A High-Performance 64-bit Adder Implemented in Output Prediction Logic

Sheng Sun, Larry McMurchie and Carl Sechen
University of Washington, Seattle, Washington, USA
larry@ee.washington.edu / sechen@ee.wasington.edu

Abstract

Output Prediction Logic (OPL) is a technique that can be applied to conventional CMOS logic families to obtain considerable speedups. When applied to static CMOS, OPL retains the restoring character of the logic family. Speedups of 2X to 3X over (optimized) conventional static CMOS are demonstrated for a variety of circuits, ranging from chains of gates, to datapath circuits, and to random logic benchmarks. Such speedups are obtained using identical netlists without remapping. When applied to pseudo-nMOS and dynamic families in combination with remapping to wide-input NORs, OPL yields even greater speedups over static CMOS. In this paper we present a novel 64-bit adder design implemented in OPL, using a combination of 8-bit Carry Look Ahead and Carry Select. The very fast wide-input OPL NORs enabled the use of 8-bit CLA units instead of the usual 4-bit. Using a process-independent metric for comparison, this adder is twice as fast as previously published 64-bit adders.

I. Introduction

Dynamic circuit families such as domino [1] are commonly used in today's high-performance microprocessors [2][3][4][5] for obtaining timing goals that are not possible using static CMOS circuits. Their increased performance is due to reduced input capacitance, lower switching thresholds, and circuit implementations that typically use fewer levels of logic due to the use of efficient and wide complex gates. It has been shown that dynamic logic can be used to realize average speed improvements of about 60% over static CMOS for random logic blocks when using synthesis tools tailored specifically for dynamic logic [6].

However, dynamic circuits have notable disadvantages. In the case of domino, logic must be mapped to a unate network, which usually requires duplication of logic. Perhaps the main disadvantage going forward is its increased noise sensitivity (compared to static CMOS). The only way to increase its noise margin is to sacrifice some of its performance gain. How to retain the good attributes of static CMOS, namely high noise immunity and easy technology mapping, while obtaining greater speed, is an elusive goal.

Output prediction logic (OPL) is a new technique that can be applied to a variety of inverting logic families to increase speed while retaining the attributes of the underlying family [17]. Given a circuit constructed of inverting logic gates, in the worst case for any circuit path, the output of every gate in the path will have to switch logic values. By correctly predicting exactly one half of the gate outputs, OPL obtains significant speedups (at least 2X) over the underlying logic families (e.g. static CMOS, pseudo-nMOS and dynamic logic). In other words, for an OPL circuit, in the worst case only every other gate output in any circuit path will have to switch logic values.

When applied to static CMOS, OPL yields circuits that are typically 2 to 3 times faster than conventional static CMOS implementations. Although OPL employs clocks, OPL-static is inherently restoring logic. OPL-static is also highly tolerant to clock skew, guaranteeing functionally correct results regardless of skew. Additionally, OPL-static uses the same synthesis tools as static CMOS (e.g. Synopsys). OPL can be applied to the same netlists as conventional static CMOS with a simple cell-for-cell substitution.

For the efficient implementation of wide NOR gates, designers often choose gates from pseudo-nMOS or dynamic logic families. OPL can be applied to these families as well. In a later section we will describe a 64-bit adder whose architecture exploits high-speed wide OPL NOR gates.

In Section II, we introduce OPL and show how it is applied to static CMOS circuits. In Section III we apply OPL to pseudo-nMOS and dynamic circuit styles. A means of generating the required clocks is outlined in Section IV. Section V quantifies the performance enhancement obtained with OPL when applied to chains of gates. Section VI describes how to apply OPL to random logic and gives results on selected ISCAS benchmarks. Section VII describes our 64-bit adder architecture in de-

1522-869X/01 $10.00 © 2001 IEEE

tail and compares its performance with other adders. We present conclusions in Section VIII.

II. Output Prediction Logic Applied to Static CMOS

In static CMOS logic, every gate is an inverting logic gate. Because of this inverting property, in the worst case every output on a circuit path must fully transition from 0 to 1, or 1 to 0 in the worst case. This is shown in Fig. 1, where we assume the primary input transitions high. This

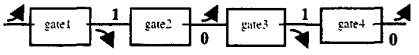

Figure 1. Static CMOS worst-case behavior.

is why static CMOS is inherently slow. A circuit designer must take into account this worst-case delay scenario for a static CMOS path.

Output Prediction Logic (OPL) greatly reduces the worst-case behavior of a circuit path. OPL predicts that every inverting gate output on a circuit path will be a

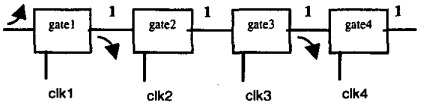

Figure 2. OPL predicting ones

logic one after the transitions are completed. Since all gates are inverting, as in static CMOS, the OPL predictions will be correct exactly one-half the time. Every other gate will not have to make any transition (see Figure 2).

There is, however, a key problem with this idea. A one at every output (and therefore input) is not a stable state for an inverting gate. The one will erode (possibly going to zero) in the latter gates of a critical path. The solution to this problem is to disable each gate with a clock, in which case ones at inputs and a one output is no longer a contradiction for an inverting gate. The gates remain disabled until their inputs are ready for evaluation. In this manner, predicted output values are maintained until new input values dictate otherwise. Successive

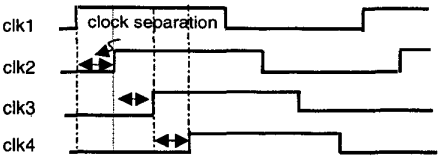

Figure 3. Clocking.

clocks are delayed by a clock separation as shown in Fig. 3.

A pre-charge-high static CMOS inverting gate implementing the above idea is shown in Fig. 4. When the

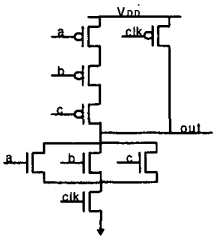

Figure 4. OPL-static CMOS NOR3.

clock (*clk*) is low, the gate is disabled, with the output being charged to a logic one. When the clock goes high, the (enabled) gate becomes a conventional static CMOS gate.

It is worth pointing out that the gate in Fig. 4 is actually a dynamic CMOS NOR3 gate with a complex (dual) keeper, rather than the usual half-latch keeper employed in typical domino circuits, as classified in [6].

While an actual circuit essentially follows this desired behavior, there are important nonidealities. Fig. 5 shows a chain of 3 OPL-static inverters and Fig. 6 shows

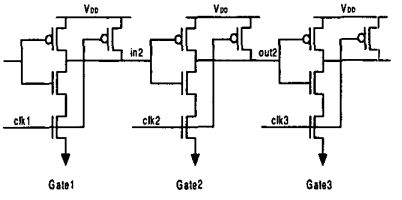

Figure 5. Chain of 3 OPL-static inverters.

the behavior of gate 2 as the clock arrival is varied. First, consider the case where the input to gate 2 in Fig. 5 is low, and therefore gate 2's output should remain high. If the clock arrives (goes fully high) at gate 2 after its input becomes stable at its low value, and if the clock to gate 3 is still low, gate 2's output will stay high at the precharged (predicted) value. If the clock arrives at gate 2 while its input is settling, a small glitch occurs, as shown

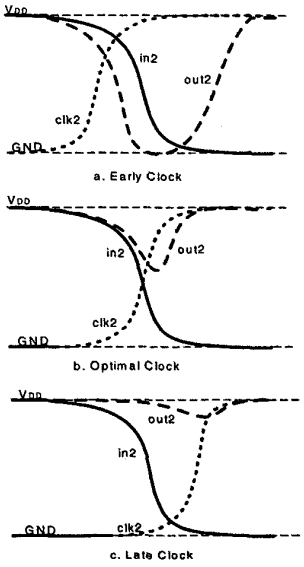

Figure 6. Dependency of gate output upon clock arrival for gate 2 of Fig. 5.

in Fig. 6c. If the clock is earlier yet – when its 50% point occurs at the same time as the 50% falling transition point of the input to gate 2 -- the still falling (but not yet fully zero) inputs will cause a bigger glitch at the output of gate 2 as shown in Fig. 6b. If the clock is even earlier yet, the precharged (predicted) value is completely lost, as shown in Fig. 6a.

The magnitude of the glitch is also enhanced by Miller kickback capacitance from load gate 3. The kickback is caused by one or both of: 1) the rapid discharge of the source of the load transistor by the evaluate transistor immediately below it, and 2) a discharge of the output node of gate 3. In each case, the effective capacitance of the gate of the load transistor has increased, stealing charge from gate 2's output node.

If we view the glitch as highly undesirable, then we need to have the rising clock (or evaluation edge) arrive after the inputs to a gate have fully settled. If this is done for all gates, then the glitch will be very small. However, in so doing, we have created what is commonly called a clock-blocked circuit, in that the throughput of the circuit is limited by the clocks and not the data. Hence, the speedup achieved may not be impressive.

At the opposite extreme, if the glitches are all the way to zero as in Fig. 6a, the throughput of the circuit is data limited. In this case, the precharge (predicted) values are completely lost and there is no reason to believe that any speedup over a conventional static CMOS circuit would be achieved.

On the other hand, if we allow a moderate amount of glitching as in Fig. 6b, then we can have the evaluation clock edges arrive somewhat earlier than the corresponding gate inputs. In this manner, the circuit is operating at the confluence of data-limited and clock-blocked behavior.

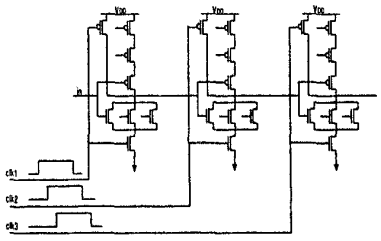

Figure 7. Chain of 3 OPL-static NOR3 gates.

Not surprisingly, we have found that there is an optimal point between the two extremes (fully clock-blocked and fully lost precharge values). As we shall see below, the minimum delay occurs when a modest amount of glitch occurs, as shown in Fig. 6b.

Fig. 7 shows a chain of OPL-static NOR3 gates. A

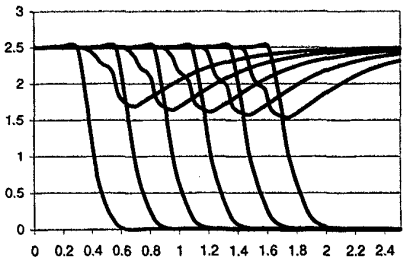

Figure 8. Waveforms (volts vs. ns) for OPL-static NOR3 chain at a clock separation of 0.13ns. The leftmost waveform is the controlling input.

chain of length 10, each gate having a fanout of four identical gates was simulated using parameters from the 0.25 micron 2.5V TSMC process [8]. An optimal clock separation was determined by sweeping over the clock separation in Hspice to find the separation (0.13ns) that gave the minimum delay. Waveforms for gate outputs at this separation are shown in Fig. 8. We see half the outputs falling to zero and the other half dipping (or glitching) and then rising back up to V_{DD}, as seen in Fig. 6b. Note that the evaluation of successive gates overlaps considerably. The objective of OPL is to control this overlap so that low-going gates get a headstart in evaluating, while high-going gates do not glitch excessively. If the pull-up and pull-down strengths are the same, one would expect the minimum overall delay should occur when a high-going output dips to a voltage intermediate between zero and V_{DD}. This voltage should be near the maximum gain point, where a small change in the input will cause a large change in the output. By controlling the clock separation we effectively position the output near this critical voltage. This contrasts to the normal operation in static CMOS, where gates begin evaluation at either zero or V_{DD} where the gain is the smallest.

Positioning gate outputs at their maximum gain point in order to increase speed has been used previously in a limited context. Zhu and Carlson describe such a method called Critical Voltage Transition Logic (CVTL) [9], applicable only for inverters. In CVTL a chain of pseudo-nMOS inverters is precharged low, then allowed to float simultaneously to a critical voltage (the point of maximum gain). Propagation delay is greatly reduced by this "preconditioning" of gate outputs. Unfortunately, this scheme depends on a very delicate balancing of loading and drive strengths between stages in order for the preconditioning state to hold. In a chain of arbitrary gates, outputs will invariably decay from a precharged value unless explicitly prevented by a method such as the OPL delayed clocks.

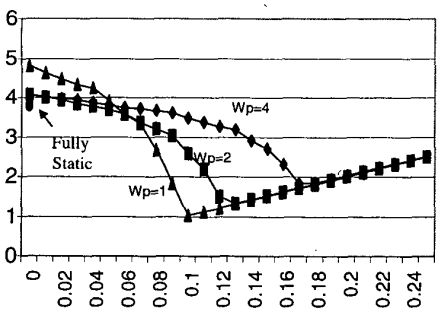

Figure 9. Total delay (ns) vs. clock separation (ns) for OPL-static NOR3 chains.

The dependency of total delay upon the clock separation can be seen clearly from Fig. 9. We show three curves, corresponding to OPL-static chains with different pMOS device sizes (W_p) in the pull-up network (pull-down devices were all sized with W_n=2μm). At zero separation, we have the case where every gate is precharged high and allowed to float at the same time. Nearly all the gates (except those near the beginning of the chain) will decay to alternating 1's and 0's before having to make a full swing to the opposite rail. Note that as the clock separations are increased from zero, more of the gate outputs approach an intermediate voltage before correcting. At the minimum in each curve, the clock separation is the effective gate delay. Eventually, as the clock separation continues to increase, the circuit (in effect) becomes clock-blocking and the delay increases linearly with clock separation.

The W_p=4μm curve corresponds to the same W_p and W_n as in the fully static gate. The delay at zero separation is very close to that of the fully static gate (4.0ns vs. 3.8ns). As one would expect, the total delay at the minimum of this curve is about half the delay at zero separation (1.9ns vs 4.0ns). We can decrease the width of the pMOS devices in the pull-up network and thereby decrease the internal loading, speeding the gates up. As we decrease W_p from 4μm to 2μm, then to 1μm, we can see the minimums in the curves decrease. However, as we decrease this size we also reduce the ability of the gates to pull up and recover from a glitch. This results in a very steep rise in the delay-separation curve before the minimum point; the circuit will be sensitive to clock skew in this region.

Note that highly accurate clocking is not required to achieve high speedups over fully static gates. In the W_p=2μm case, a 10% error in the overall average clock results in a speedup (over fully static) of 2.5X vs 2.8X if the clock were positioned at the exact minimum.

III. Output Prediction Logic Applied to Pseudo-nMOS and Dynamic Design

The OPL technique can be applied to pseudo-nMOS as well as dynamic circuits. A pre-charge-high pseudo-nMOS gate is shown in Fig. 10. When the clock (*clk*) is low, the gate is disabled, with the output being charged to a logic 1. When the clock goes high, it becomes a pseudo-nMOS gate. The pull-up serves both to precharge the gate and correct a high output when it glitches. This pMOS device is sized in accordance with the pull-down stack to yield an appropriate output-low voltage. Note that the output-low voltage can be set closer to zero than for conventional pseudo-nMOS since pull-up delay is less of a concern, thus lowering static power dissipation (as will be shown later). The behavior of the gate is similar to that of pseudo-nMOS. Once *clk* goes high, we would expect this gate to outperform OPL-static for wide input NORs, where the pull-up chains are not as effective as a single pull-up device in correcting a high output that has glitched.

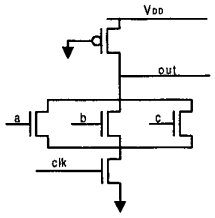

Figure 10. OPL-pseudo-nMOS NOR3.

We also tested the OPL concept with dynamic logic gates. As shown in Fig. 11, an OPL-dynamic gate looks exactly like a domino gate, but with the output inverter missing. Note that the gate precharges high, and that the keeper, if sized sufficiently large, will enable the output node to recover from glitches. If the clock arrives too early (keep in mind that the inputs precharge high), a gate may glitch so much that the keeper shuts off, causing the output voltage to remain at a value possibly well below V_{DD} (or even zero). Thus, in contrast to OPL-static and OPL-pseudo, and similar to conventional domino gates, OPL-dynamic gates can fail functionally.

Note that the OPL-dynamic gate is very different from a conventional domino gate, as it does not have a following inverter. Domino circuits are positive unate

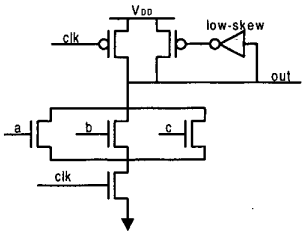

Figure 11. OPL-dynamic NOR3.

and may have circuit paths that require every gate to discharge. Such circuits will therefore be slower than OPL-dynamic where one can take advantage of the alternating nature of the logical output values of the gates on circuit paths to speed up the circuit. Domino circuits also generally require logic duplication to map to positive unate functions, in contrast to OPL circuits. Note that there are never any static inverters, nor any other type of static CMOS gate, in an OPL-dynamic circuit. In order to achieve the benefit of only having every other gate output requiring a transistion, any necessary inverters must be implemented as OPL-dynamic inverters.

IV. Generation of Clocks

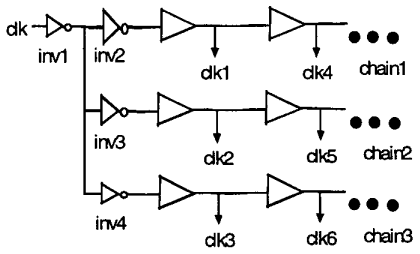

Figure 12. Generation of the clocks.

The fast speed of OPL logic requires the clocks to be separated by a small amount, typically less than a buffer delay. Thus, a normal chain of delay buffers is not sufficient to generate these clocks since each clock will be separated by a buffer delay that is more than a gate delay. The required clock separations can be generated by using

several buffer delay chains as shown in Fig. 12. For example, if we want a clock separation equal to 1/3 of a buffer delay, then inverters *inv2* and *inv3* are sized such that *chain2* lags *chain1* by 1/3 of a buffer delay. Therefore, *clk2* is 1/3 of a buffer delay behind *clk1* and *clk5* is 1/3 of a buffer delay behind *clk4*. Similarly, *inv3* and *inv4* are sized such that *chain3* lags *chain2* by 1/3 of a buffer delay. Thus, *clk3* is 1/3 of a buffer delay behind *clk2* and *clk6* is 1/3 of a buffer delay behind *clk5*. Since *clk4* is one buffer delay behind *clk1* and *clk3* is 2/3 of a buffer delay behind *clk1*, *clk4* is 1/3 of a buffer delay behind *clk3*. As a result, all clocks are separated by 1/3 of a buffer delay. To achieve arbitrary clock separations, we modify the buffer delay and size *inv2*, *inv3* and *inv4* accordingly, and/or increase or decrease the number of chains.

V. OPL Performance on Chains of Gates

To determine the performance possible with OPL, we simulated critical paths consisting of 10 identical gates, each gate in the path driving a load of four identical gates. We used nominal simulation parameters for the TSMC 2.5 volt, 0.25-micron process with a drawn channel length of 0.30 microns [8]. The delay results in Table 1 compare static CMOS, OPL-static, OPL-pseudo-nMOS (OPL-pseudo) and OPL-dynamic. The numbers in parentheses are the speeds relative to the static CMOS chain. From the INV data, it is apparent that the fanout-of-four static CMOS inverter delay for this process is 160ps.

Table 1. Delays(ns) for chains of 10 gates (FO of 4).

Chain Type	Static CMOS	OPL Static	OPL Pseudo	OPL Dynamic
INV	1.62 (1.0)	0.43 (3.77)	0.42 (3.86)	0.43 (3.77)
NOR3	3.83 (1.0)	1.34 (2.86)	0.71 (5.39)	0.76 (5.04)
NAND2	2.45 (1.0)	0.94 (2.61)	0.93 (2.63)	1.02 (2.40)
NAND3	3.32 (1.0)	1.44 (2.31)	1.54 (2.16)	1.54 (2.16)
NAND4	4.24 (1.0)	1.97 (2.15)	2.16 (1.96)	2.15 (1.97)
AOI22	4.75 (1.0)	2.13 (2.23)	1.81 (2.62)	1.80 (2.64)
AOI222	6.75 (1.0)	3.04 (2.22)	2.63 (2.57)	2.49 (2.71)
Average	(1.0)	(2.59)	(3.03)	(2.96)

The nMOS devices for all versions of the above gates were sized to have an effective pull-down width of 2 μm. Thus, if the pull-down portion of a gate had stack of k transistors in series, the size of the transistors was $2k$ μm. The pMOS transistors for the static CMOS gates were uniformly sized by sweeping their size versus overall delay for the chain of 10 gates, and then selecting the size that minimized the worst-case delay for the chain. As explained in Section II, the sizes of the pMOS devices in the OPL-static gates were selected to obtain high speed as well as suitably fast recovery from glitches at the output nodes. We used 2 μm for all pMOS devices except for NAND3 and NAND4 gates, which used 3 μm. For OPL-pseudo, 1.4μm was used for the pMOS pull-up device. The keeper pull-up pMOS device was 2 μm for OPL-dynamic.

On average, OPL-static chains are 2.6 times faster than static CMOS and OPL-pseudo chains are 3 times faster than static CMOS. In fact, OPL-pseudo NOR3 chains are 5.4 times faster than static CMOS, and INV chains are 3.86 times faster. NOR gates with an arbitrary number of inputs are extremely fast with OPL-pseudo. On the other hand, OPL-static is faster for NAND gates than OPL-pseudo. OPL-dynamic has about the same performance as OPL-pseudo. The results so far have been for homogeneous chains of gates.

Table 2. Delays for heterogeneous chains of 8 gates.

Logic Family	Delay	Speedup
Static CMOS	2.13ns	1.0
OPL Static	910ps	2.34
OPL Pseudo	650ps	3.28
OPL Dynamic	688ps	3.10

We also constructed a chain of eight heterogeneous gates, which were (in order): NOR3, NAND3, AOI22, INV, INV, NOR3, NAND3, and AOI22. Each gate drives a load of four identical gates. The device sizes used were exactly those selected for the uniform chains. Having the gates so ordered means that each gate type will have to pull down once and stay high once. Table 2 shows the results. While it might be expected that tailoring the clock separations to the specific gate types would yield the best results, we have found that not to be true. A uniform clock separation, which is inherently easier to generate, yields results equivalent to specially tailored clock separations. The speedup obtained for the heterogeneous chain is quite similar to that obtained for the average homogeneous chain.

We also compared the total energy consumption for static CMOS, OPL-static and OPL-pseudo (including precharging energy), as well as OPL-dynamic. Table 3 shows that the average energy consumption of OPL-static gates is around 1.3 times of that for static CMOS. This energy includes the energy needed to generate the clocks, precharge outputs, and any energy consumed during evaluation. OPL-pseudo gates consume about 2.2 times as much energy as static CMOS gates, on average. OPL-

dynamic consumes less energy than OPL-pseudo and offers similar speed.

Table 3. Energy consumption(in pJ) for chains of 10 identical gates. Numbers in parentheses are relative to static CMOS.

Chain Type	Static CMOS	OPL Static	OPL Pseudo	OPL Dynamic
INV	2.00 (1.0)	3.80 (1.90)	4.97 (2.49)	4.41 (2.21)
NOR3	3.19 (1.0)	4.45 (1.39)	6.07 (1.90)	4.47 (1.40)
NAND2	3.83 (1.0)	5.00 (1.31)	8.39 (2.19)	5.60 (1.46)
NAND3	6.23 (1.0)	6.66 (1.07)	12.7 (2.04)	7.51 (1.21)
NAND4	8.65 (1.0)	12.7 (1.47)	19.3 (2.23)	10.0 (1.16)
AOI22	6.13 (1.0)	6.31 (1.03)	12.8 (2.09)	7.01 (1.14)
AOI222	7.08 (1.0)	7.70 (1.09)	16.7 (2.36)	8.09 (1.14)
Average	(1.0)	(1.32)	(2.19)	(1.39)

We also investigated the dependence of total delay of the chains upon clock skew. Because the total delay is most sensitive to the skew of clocks controlling gates near the beginning of the chains, we introduced a variable skew of up to ±30ps into the clock controlling the second gate in the chain. The resulting percentage increase in total delay of the chain is shown for NOR4 and NOR8 in Fig. 12a. Without any skew the total delay of the NOR4 chain is 743ps and the delay of the NOR8 chain is 1139ps. The skew magnitude is a higher percentage of the average gate delay (and hence the average clock separation) for the NOR4 chain. Therefore, it is not surprising that larger gates show considerably less dependence upon skew than smaller gates. As we shall see later, this characteristic influenced our choice of adder architectures.

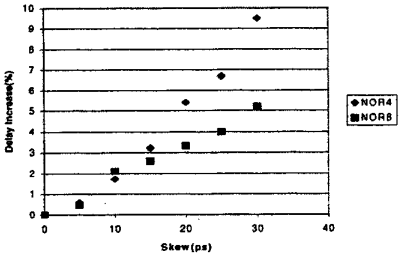

Figure 12a. Dependence of total delay upon clock skew for NOR4 and NOR8

VI. Application of OPL to Random Logic

We implemented five ISCAS benchmark circuits in both static CMOS and OPL-static. The networks were obtained with Synopsys, using a library with functionality similar to lib2.genlib (26 logically different cells) with the addition of inverters and buffers of various drive strengths. All possible Synopsys scripts were used and the fastest static CMOS implementation was selected. The resulting network was then used for both static CMOS and OPL-static experiments. Note that the networks used in both experiments were identical; only the cell implementations were different. Both implementations were analyzed using Pathmill.

For OPL-static, all paths with delays within 10% of the critical path were selected in Pathmill. The pull-up delays of these gates were artificially set to be small in Pathmill to properly mimic the behavior of OPL-static paths in which the primary delay contributor is the pull-down delay of every other gate in a path.

We computed the logic level of each gate and assigned clock i to all gates at level i. The resulting set of OPL-static paths was then simulated using Hspice. The clock separation was varied and the separation selected that gave the minimum worst-case delay over all paths.

Table 5. Results for ISCAS benchmarks

Circuit	Static	OPL-static	
	Delay (ns)	Delay(ns)	#Clocks
t481	0.91 (1.0)	0.46 (1.98)	7
term1	1.38 (1.0)	0.70 (1.97)	11
x3	2.20 (1.0)	0.67 (3.28)	10
rot	2.19 (1.0)	1.05 (2.09)	17
dalu	2.35 (1.0)	0.96 (2.45)	15
Avg	(1.0)	(2.35)	

Results comparing OPL-static with conventional static CMOS are shown in Table 5. On average, OPL-static is 2.35 times faster than static CMOS. Circuit *x3* exhibits a particularly large speedup over static because of the preponderance of NOR3, NOR4 and inverters on critical paths. Even greater speedups over static CMOS would be possible for these circuits if they were mapped to wider gates (e.g. NORs). Technology mapping tools targeting such wide gates have been demonstrated for CD domino logic [6,7].

It is noteworthy that OPL-static is consistently at least 2.35 times faster than static CMOS (on average) for homogeneous chains of gates, heterogeneous chains of gates, and random logic blocks.

VII. 64-bit Adder

Our goal was to design the fastest adder possible using OPL. Our selection of the adder architecture was heavily influenced by the fact that OPL offers the maximum speed advantages for NOR gates, and in particular, for fairly wide NOR gates. Since both the CLA (Carry Look-Ahead) and CSA (Carry Select Adder) techniques are readily implemented using only NOR gates, we found that an OPL-based architecture based on these two techniques yielded the highest possible speed. Not surprisingly, the CLA and CSA techniques have formed the basis for many high-speed adders, such as the 64-bit adders by D. Harris [10], S. Naffziger [11] and Lynch-Swartzlander [12].

The adders in [10-12] were all based on 4-bit CLA units, which is usually the optimal tradeoff between the depth of the CLA tree and the number of series transistors in one static CMOS gate. However, 4-bit CLA structures only generate carries at 16-bit boundaries as in [10] and [11], requiring 16-bit sub-adders for Carry Select units. Alternatively, one can use a special technique like "spanning" as in [12] to generate 8-bit boundary carries. All of these structures are quite complex.

Increasing the number of bits handled by a CLA unit (e.g. from 4 to 8) will result in the use of fewer logic levels and a more regular design and layout. Of course, an 8-bit CLA unit is impractical in static CMOS since it requires 8 series transistors for static CMOS gates. However, by applying DeMorgan's Law to the CLA equations, we can take advantage of the effectiveness of wide NOR gates in OPL [7]. We therefore obtain the regular structure shown in Fig. 13. It has only $\log_8 64$, or two levels of carry look-ahead. Using 8-bit blocks does reduce the number of logic levels, but it may not reduce the total delay since each CLA level is slower, with bigger gates and more fanout.

We also built a 64-bit, OPL-based adder using 4-bit CLA units, for comparison. It turned out to be slower than the architecture presented here, and was much more difficult to lay out, leading to long wires. Besides the layout efficiency, the 8-bit architecture has somewhat slower gates (NOR8's), which have a larger optimal clock separation, as shown earlier. Therefore, also as shown earlier, the NOR8-based design is more clock-skew tolerant.

Naffziger's adder uses Ling's equations [11] to reduce the overall depth of the CLA tree by one logic level. However, Ling's equations are only applicable for 4-bit units, in that the equations already have 8 terms [11][13]. We also tried Ling's equations for our 4-bit architecture but they turned out to not be useful for a NOR-gate based approach, since that would require both polarities of the primary inputs.

Among the family of prefix-computation adder architectures [14], Kogge-Stone [15][16] is the fastest. It generates carries for each bit so no carry select is needed. With the original binary Kogge-Stone scheme, the depth is $\log_2 64 = 6$. The 4-bit and 8-bit versions have depth $\log_4 64 = 3$ and $\log_8 64 = 2$, respectively. However, they introduce many more CLA units, while dramatically increasing wires and fanouts. We were not able to find any advantage of the Kogge-Stone scheme for our OPL-based approach since it would require the same number of logic levels and would not provide any advantage in terms of area.

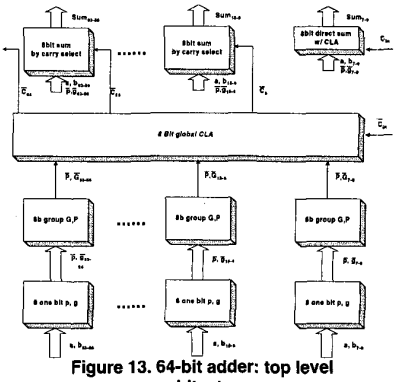

Figure 13. 64-bit adder: top level architecture

We now examine the major blocks in our 64-bit adder shown in Fig. 13. The global carries generated at each 8-bit position are used to select between two sets of sums pre-computed by two 8-bit adders (Fig. 14). The 8-bit adder is also a CLA-based adder (Fig. 15). The CLA equations are transformed into NOR gates (Fig. 16). Each CLA level consists of two NOR stages, which we have found to be considerably faster than a single large AOI gate performing the same function. Signals are propagated in negative polarity instead of positive. Figure 17 shows the critical path of the adder.

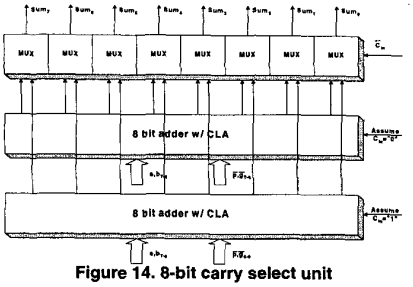

Figure 14. 8-bit carry select unit

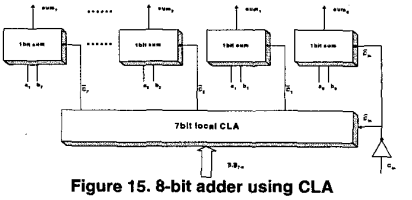

Figure 15. 8-bit adder using CLA

$$\overline{C_1} = g_0 + p_0 + C_{in}$$

$$\overline{C_2} = g_1 + p_1 + g_0 + p_1 + p_0 + C_{in}$$

$$\overline{C_3} = g_2 + p_2 + g_1 + p_2 + p_1 + g_0 + p_2 + p_1 + p_0 + C_{in}$$

.....

$$\overline{C_7} = g_6 + p_6 + g_5 + ...p_6 + p_5 + p_4 + p_3 + p_2 + p_1 + p_0 + C_{in}$$

Figure 16. CLA equations (facilitating the use of NOR gates)

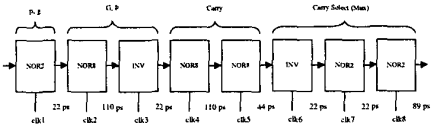

Figure 17. The adder's critical path is shown. It consists of 3 NOR8's, 3 NOR2's and two inverters.

Our 64-bit adder uses eight clock phases. Fig. 17 shows the clock separations required by each of the clock phases. As shown, the two NOR8s require a clock separation equal to 5X the smallest separation (22ps). All separations were obtained from simulations using a 0.25-micron TSMC process (with a drawn channel length of 0.30 microns).

Traditionally it has been very difficult to compare the speeds of adders (or other logic blocks) implemented using different processing technologies. However, Prof. Mark Horowitz at Stanford University recently pointed out [10] that delays could be easily normalized with respect to a process. This is done by taking the actual delay measurement and dividing it by the fanout-of-4 inverter delay for that process. It is a remarkable result that this unit-less delay value will remain unchanged if a design is migrated to a new process. We are therefore able to accurately compare the speed of our OPL-based 64-bit adder with the two fastest previously published 64-bit adders [10,11]. The results are shown in Table 6. It is apparent that our new 64-bit adder is more than twice as fast as the best previously published adders.

It should be noted that the fanout-of-four inverter delay for our 0.25um (having a drawn channel length of 0.30um) process is 162ps, as shown in the first row of Table 1. This is an extremely slow process corner, and is the fastest one we had available. It is therefore not surprising that a 0.5um process would have a fanout-of-four inverter delay that is less than ours, especially if a somewhat faster process corner is used.

Table 6.
Comparison of 64-bit high-speed adders

64-bit Adder Version	Process	Delay	Delay normalized to FO4 INV Delay
OPL	.25μm	463 ps	2.9
D. Harris, Stanford	.6μm		6.4
S.Naffziger, HP	.5μm	930 ps	7

Also of interest is how robust the delay of our adder is with respect to clock skew. Based on consultations with designers at several Semiconductor Research Corporation (SRC) member companies, we may face a worst-case skew of a clock phase by plus or minus 30ps. As shown in Fig. 18, the delay will increase to 493ps. This represents

3.04 FO4 INV delays, which is still considerably faster than previous work.

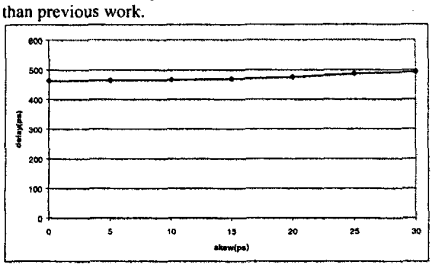

Figure 18. Dependence of adder total delay upon skew

VII. Conclusions

We presented Output Prediction Logic, a technique that can be applied to conventional CMOS logic families to obtain considerable speedups. Average speedups of 2.35X over conventional static CMOS were demonstrated for a variety of circuits, ranging from chains of gates, to datapath circuits, and to random logic benchmarks. These speedups are obtained while retaining the noise margins and level restoring nature of static CMOS as well as the same netlists. OPL has excellent noise margins since each gate is a low-skew gate, quite unlike domino logic where critical paths consist of alternating low-skew and high-skew gates. We also designed a 64-bit, OPL-based adder that was more than twice as fast as the best previously reported adders.

Acknowledgements

We would like to acknowledge the financial support provided by the Semiconductor Research Corporation, the National Science Foundation, and the NSF Center for the Design of Analog and Digital ICs (CDADIC). We would also like to acknowledge the contributions of Jovanka Ciric.

References

1. D. Harris, "Skew-tolerant circuit design," Ph. D. dissertation, Stanford University, Stanford, CA, 1999.
2. F. Klass, et. al., "A new family of semi-dynamic and dynamic flip-flops with embedded logic for UltraSPARC-III," *IEEE J. Solid-State Circuits*, vol. 34, no. 5, pp. 712-717, May 1999.
3. B. Benschneider, et. al., "A 300-MHz 64-b quad-issue CMOS RISC microprocessor," *IEEE J. Solid-State Circuits*, vol. 30, no. 11, pp. 1203-1214, Nov. 1995.
4. A. Charnas, et. al., "A 64b microprocessor with multimedia support," *ISSCC Dig. of Tech. Papers*, pp. 177-179, Feb. 1995.
5. R. Colwell and R. Steck, "A 0.6um BiCMOS processor with dynamic execution," *ISSCC Dig. of Tech. Papers*, pp. 176-177, Feb. 1995.
6. T. Thorp, G. Yee and C. Sechen, "Monotonic Logic and Dual Vt Technology", *Proc. of Int. Conf. on Computer Design (ICCD)*, Austin, TX, October 1999.
7. G. Yee and C. Sechen, "Clock-delayed Domino for Adder and Random Logic Design," Proc. IEEE Int. Conf. on Computer Design (ICCD), October 7-9, 1996, Austin, TX.
8. MOSIS, "MOSIS Parametric Test Results," http://www.mosis.org.
9. Z. Zhu and B. Carlson, "Critical Voltage Transition Logic: An Ultrafast CMOS Logic Family", Proc. IEEE Int. Conf. On Computer Design (ICCD), Austin, TX, October 1997.
10. M. Horowitz, EE271 class notes (Adders), Stanford University, 1999.
11. S. Naffziger, "A Sub-Nanosecond 0.5um 64b Adder Design," 1996 IEEE ISSCC, Digest of Technical Papers, pp. 362-363; Slide Supplement, pp. 290-291.
12. T. Lynch and E. Swartzlander, "A Spanning Tree Carry Lookahead Adder," IEEE Tran. On Computers, Vol. 41, No. 8, August 1992.
13. H. Ling, "High-Speed Binary Adder," IBM J. Research. Develop. Vol. 25, No. 3, May 1981.
14. S. Knowles, "A Family of Adders," 14th IEEE Symposium on Computer Arithmetic, Proceedings, pp30-34, 1999.
15. P. Bunyk, P. Litskevitch, "Case Study in RSFQ Design: Fast Pipelined Parallel Adder," IEEE Tran. On Applied Superconductivity, Vol. 9, No. 2, June 1999.
16. P. Kogge and H. Stone, "A Parallel Algorithm for the Efficient Solution of a General Class of Recursive Equations," IEEE Tran. On Computer, Vol. C-22, No. 8, August 1973.
17. L. McMurchie, S. Kio, G. Yee, C. Sechen, Output Prediction Logic: A High-Performance CMOS Design Technique, Proceedings of ICCD, Sept. 2000.